Introduktion til AI

MM8: Decision trees and random forest

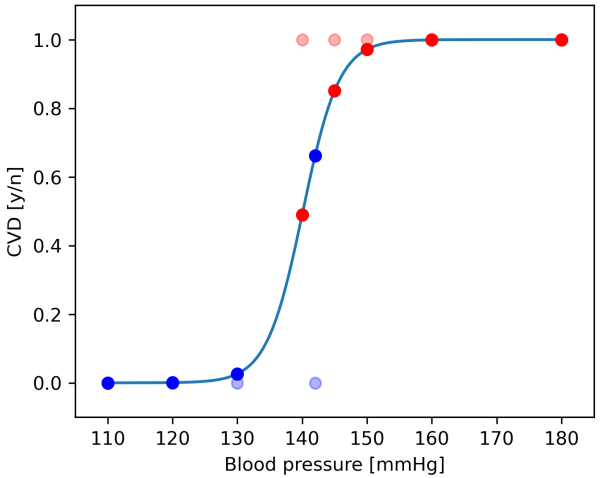
Thomas Kronborg Larsen, PhD
(tkl@hst.aau.dk)



Agenda

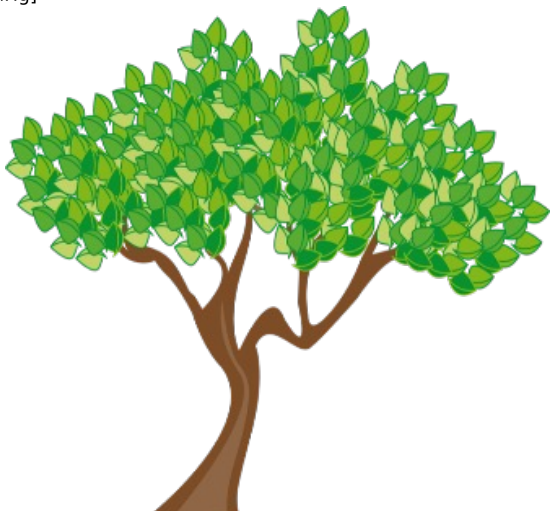
Last time we estimated parameters to predict classes using probabilities:

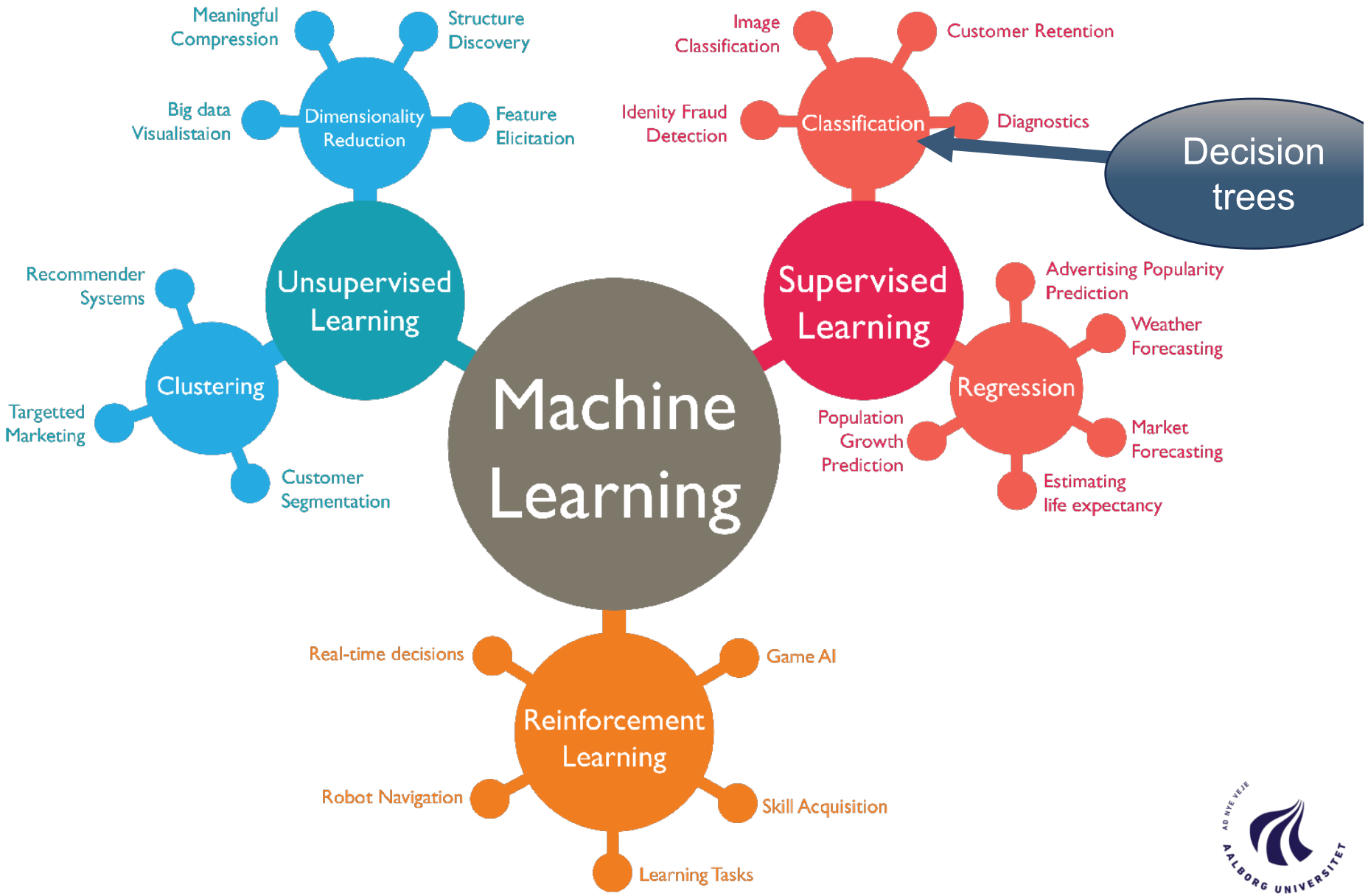
$$\hat{p} = \frac{1}{1 + e^{-(\theta_0 + \theta_1 x_1)}}$$



Now lets try another approach:

Decision trees are **non-parametric** supervised classification algorithms

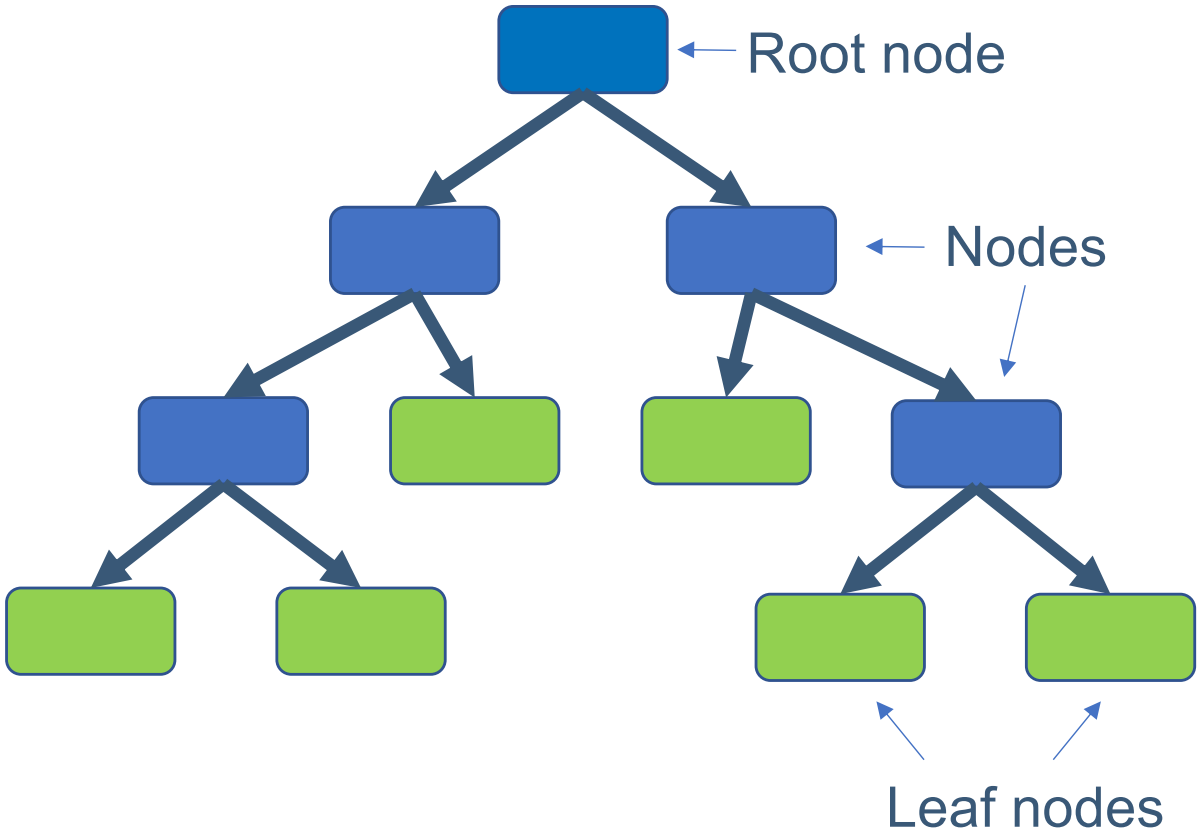



Meaningful Compression
Structure Discovery
Big data Visualistaion
Dimensionality Reduction
Feature Elicitation
Unsupervised Learning
Recommender Systems
Clustering
Targetted Marketing
Customer Segmentation
Machine Learning
Image Classification
Customer Retention
Idenity Fraud Detection
Classification
Diagnostics
Decision trees
Supervised Learning
Advertising Popularity Prediction
Weather Forecasting
Regression
Population Growth Prediction
Market Forecasting
Estimating life expectancy
Real-time decisions
Game AI
Reinforcement Learning
Robot Navigation
Skill Acquisition
Learning Tasks

Decision trees

Terminology:



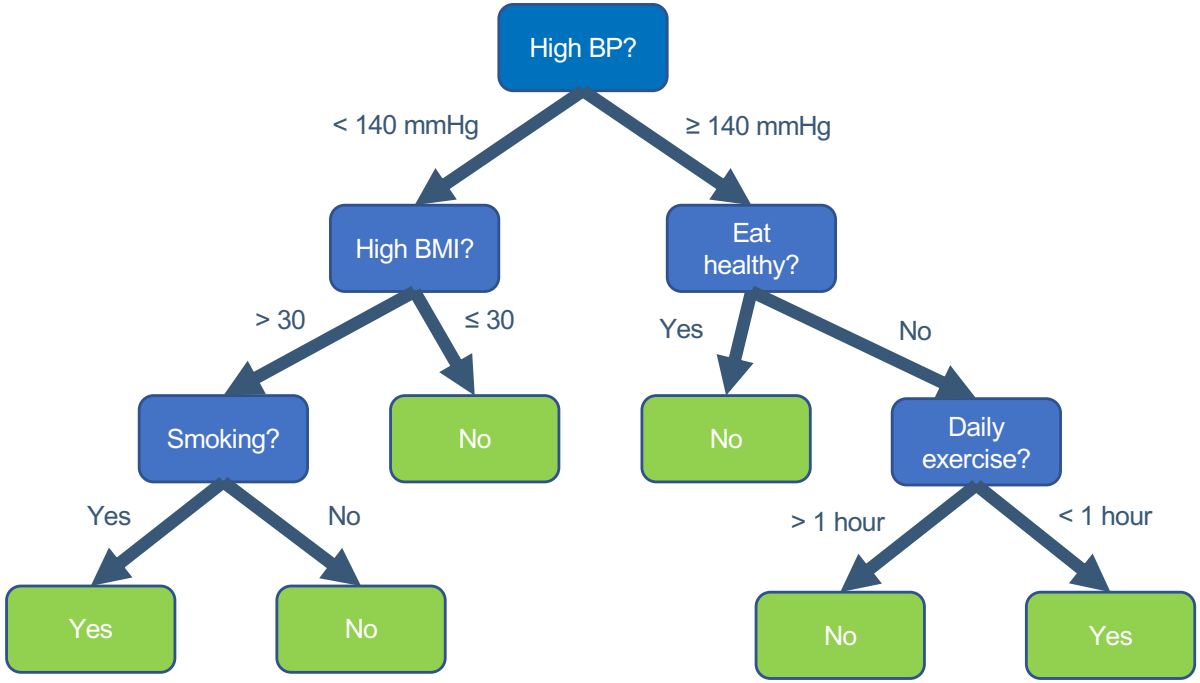


Decision trees

Terminology:



Risk of heart disease example:



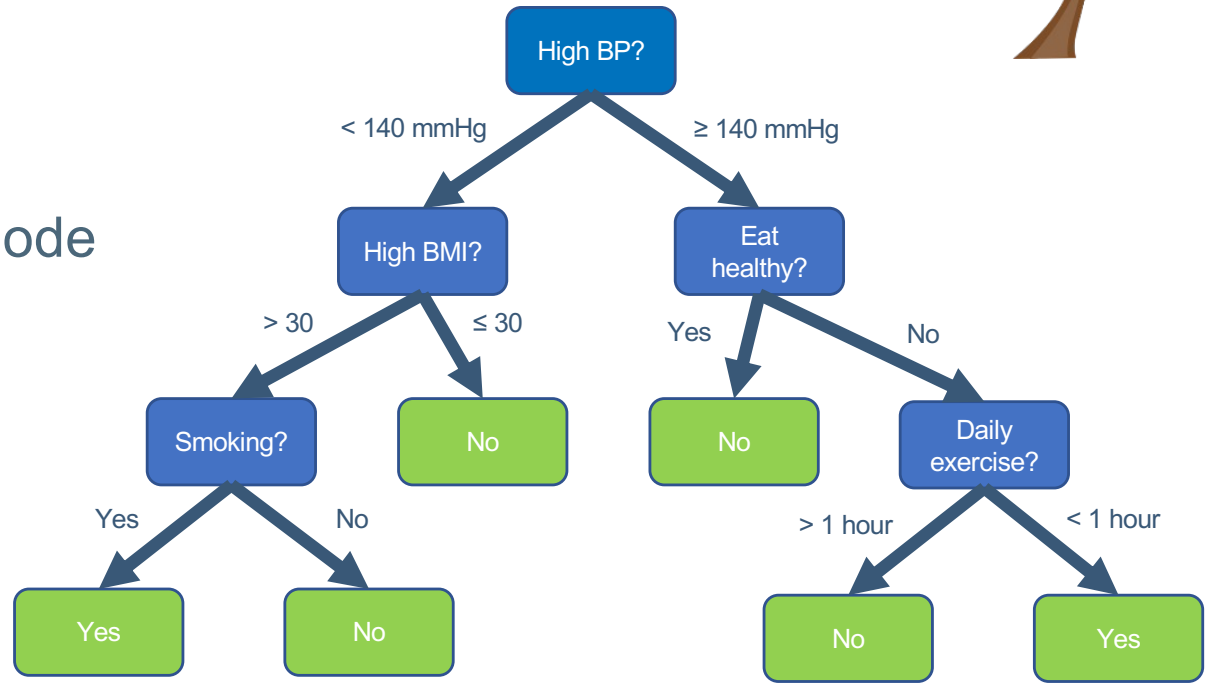
Decision trees



Classification:

- Start at root node
- Go to a descending node based on the query
- Stop when a node have no further links

Risk of heart disease example:



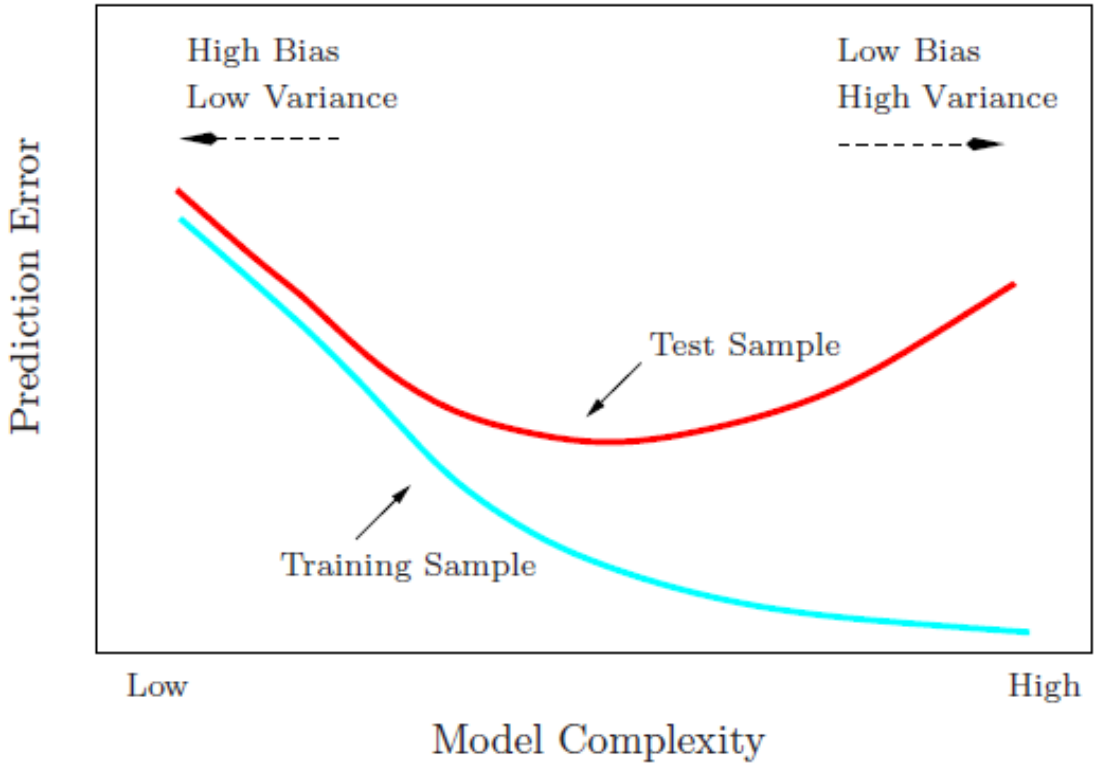
Decision trees

Classification:



- Start at root node
- Go to a descending node based on the query
- Stop when a node have no further links

Just keep splitting?



Decision trees





Can we predict good from evil?

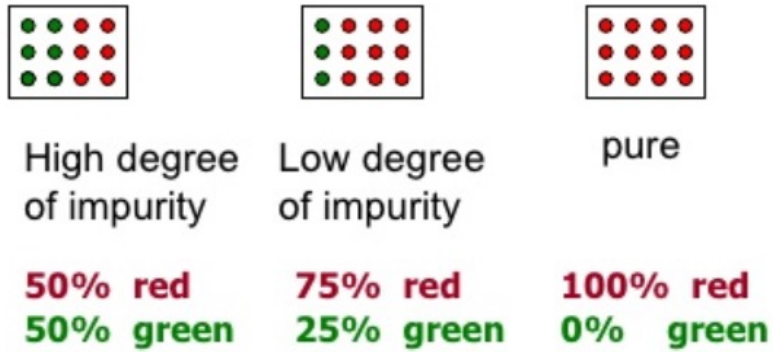
Decision trees

Determining the splits:

We need a metric to optimize purity…

Gini impurity

Entropy



Decision trees

Gini impurity:

Data: $\mathrm{m} = \{(\mathrm{x}_1, \mathrm{y}_1), \ldots, (\mathrm{x}_\mathrm{n}, \mathrm{y}_\mathrm{n})\},\ \mathrm{y}_\mathrm{i} \in \{1, \ldots, \mathrm{c}\}$ (c is the number of classes)

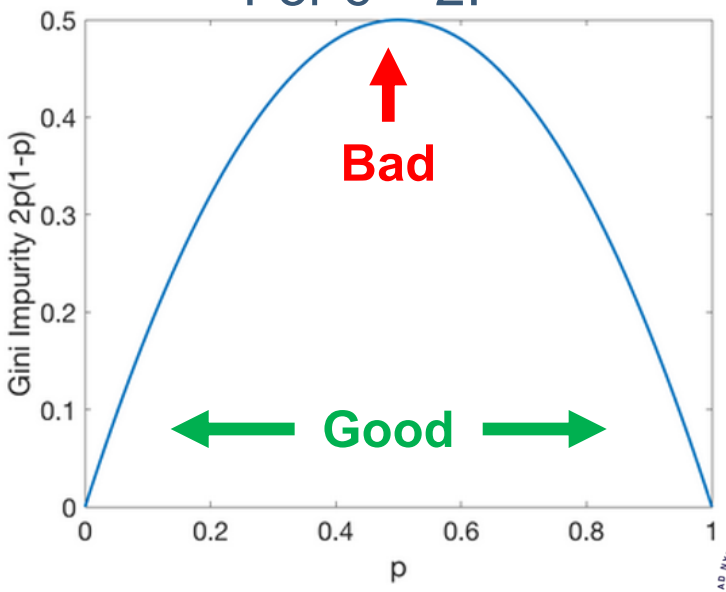
Let: $\mathrm{m}_\mathrm{k} \subseteq \mathrm{m}$, where $\mathrm{m}_\mathrm{k} = \{(\mathrm{x}, \mathrm{y}) \in \mathrm{m} \mid \mathrm{y} = \mathrm{k}\}$ (subset with same class label)

$\mathrm{m} = \mathrm{m}_1 \cup, \ldots, \cup \mathrm{m}_\mathrm{c}$

Define: $\mathrm{p}_{i,k} = \frac{|m_k|}{|m|}$

$$\mathrm{G}_\mathrm{i} = 1 - \sum_{k=1}^{c} \mathrm{p}_{i,k}{}^2$$

For c = 2:



G_i is the Gini impurity of the ith node
$p_{i,k}$ is the ratio of class k instances among the training instances in the ith node

Decision trees

Entropy:

What we dont want: $\mathrm{p}_1 = \mathrm{p}_2 = \ldots = \mathrm{p}_k = \frac{1}{c}$ (All classes equally likely)

We define impurity as how close we are to uniform: **(KL-divergence)**

The futher away from uniform, the lower the following:

$$\mathrm{H_i} = -\sum_{k=1}^{c} \mathrm{p}_{i,k} \log(\mathrm{p}_{i,k})$$

(Minimizes entropy)

Example:

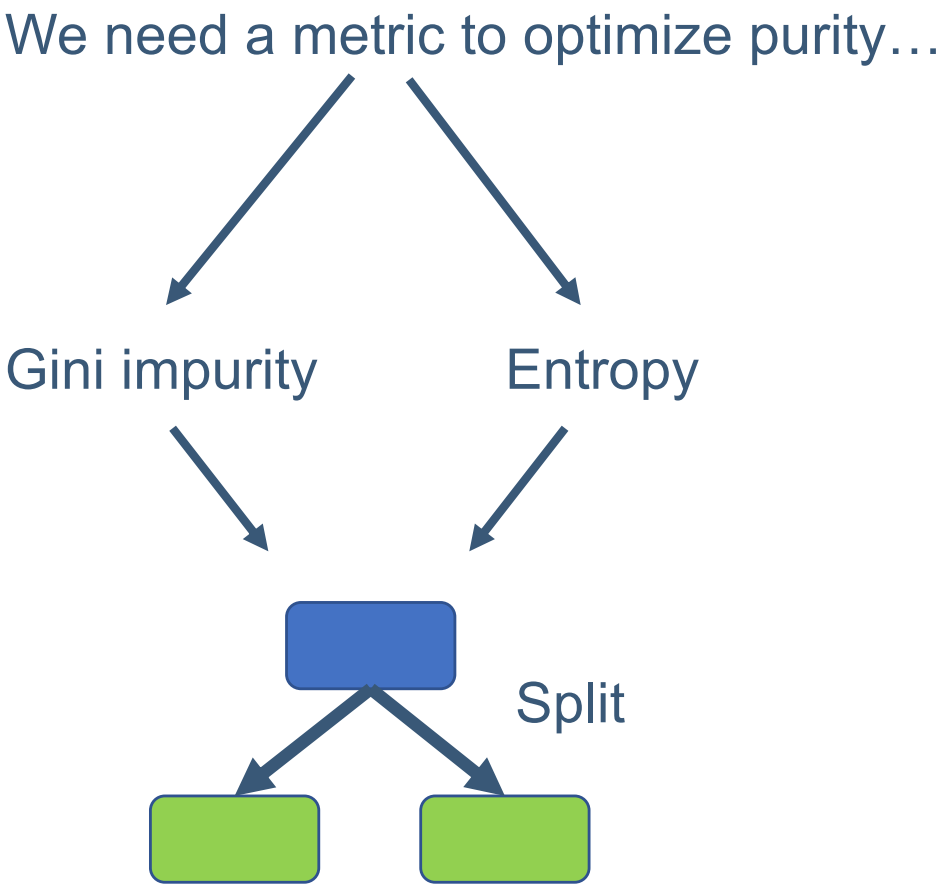
Bad split $H_i = -\left(\frac{50}{100}\right)\log\left(\frac{50}{100}\right) - \left(\frac{50}{100}\right)\log\left(\frac{50}{100}\right) = 0.30$

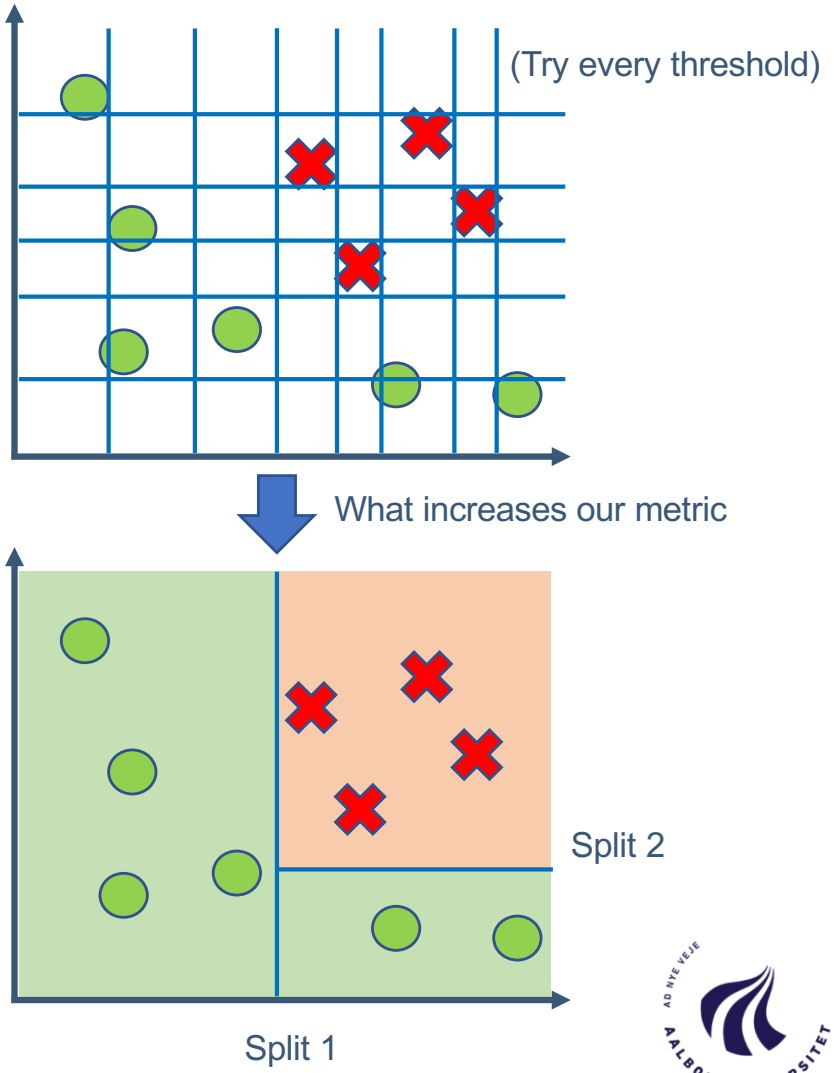
Good split $H_i = -\left(\frac{5}{100}\right)\log\left(\frac{5}{100}\right) - \left(\frac{95}{100}\right)\log\left(\frac{95}{100}\right) = 0.09$

H_i is the entropy of the ith node
$p_{i,k}$ is the ratio of class k instances among the training instances in the ith node

Decision trees

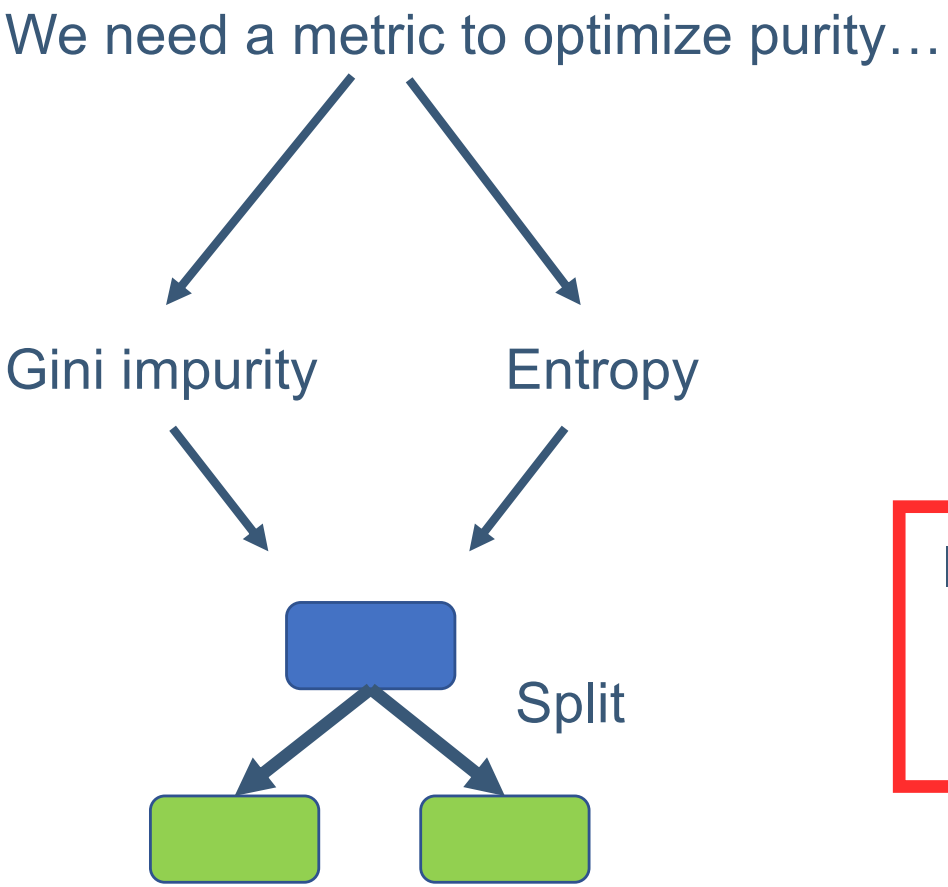
Determining the splits:

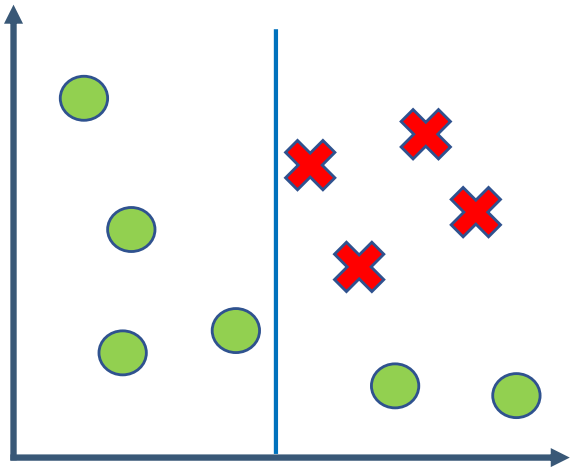




Decision trees

Determining the splits:





Impurity of split (Gini):

$$\frac{m_{left}}{m} G_{left} + \frac{m_{right}}{m} G_{right}$$

Decision trees

Using **Scikit-Learn**:

```
from sklearn.tree import DecisionTreeClassifier  # Import the Decision Tree classifier

# Load the Iris dataset
iris = datasets.load_iris()
X = iris.data[:, :2]  # Using only the first two features (sepal length and sepal width)
y = iris.target

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Create a Decision Tree classifier
decision_tree_classifier = DecisionTreeClassifier()

# Fit the Decision Tree classifier to the training data
decision_tree_classifier.fit(X_train, y_train)

# Make predictions on the test set
y_pred = decision_tree_classifier.predict(X_test)

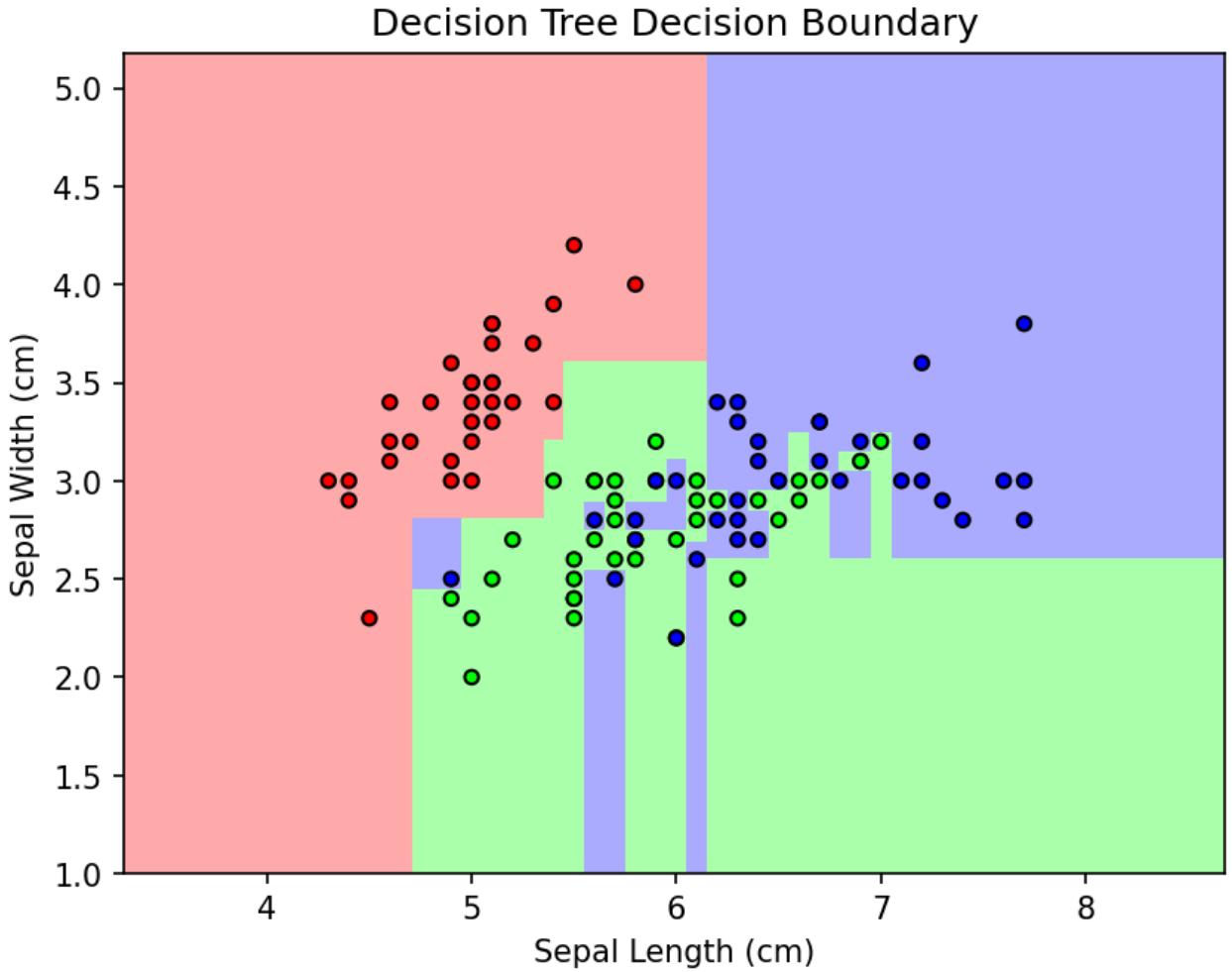
# Calculate the accuracy of the model
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy of Decision Tree: {accuracy * 100:.2f}%")
```

```
Accuracy of Decision Tree: 66.67%
```

Decision trees

Using **Scikit-Learn**:

Is anything wrong here?



Decision trees

Using **Scikit-Learn**:

```
from sklearn.tree import DecisionTreeClassifier  # Import the Decision Tree classifier

# Load the Iris dataset
iris = datasets.load_iris()
X = iris.data[:, :2]  # Using only the first two features (sepal length and sepal width)
y = iris.target

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Create a Decision Tree classifier
decision_tree_classifier = DecisionTreeClassifier(min_samples_leaf=3)

# Fit the Decision Tree classifier to the training data
decision_tree_classifier.fit(X_train, y_train)

# Make predictions on the test set
y_pred = decision_tree_classifier.predict(X_test)

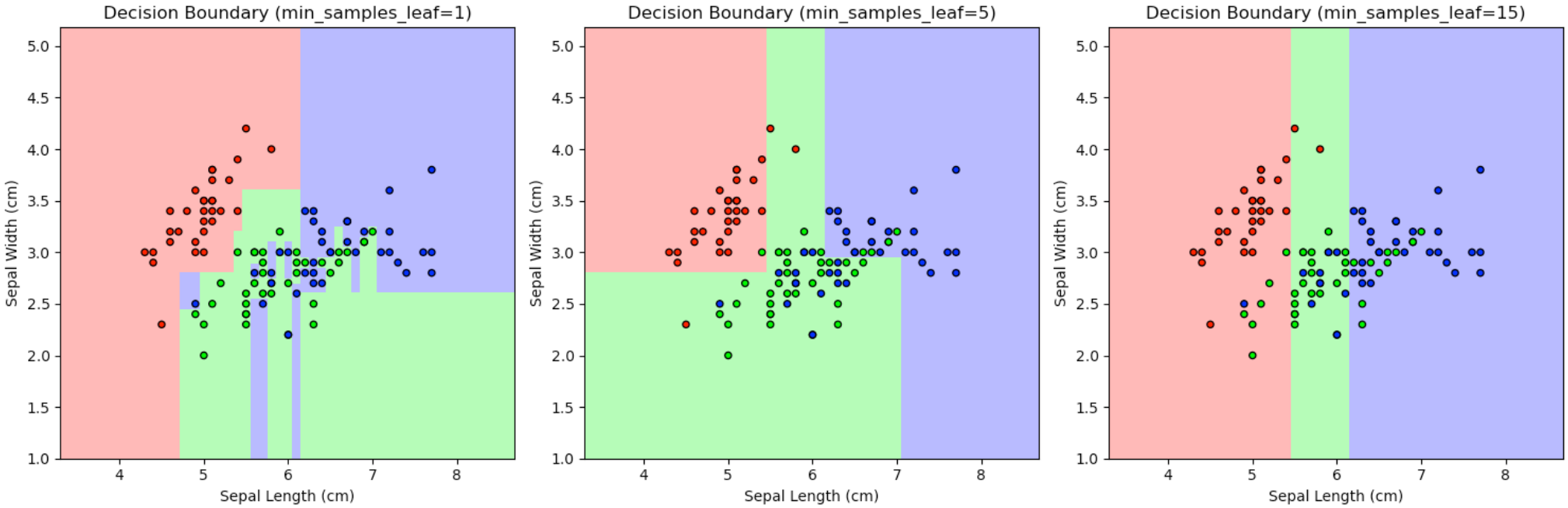
# Calculate the accuracy of the model
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy of Decision Tree: {accuracy * 100:.2f}%")
```

```
Accuracy of Decision Tree: 77.78%
```

Let's try regularization to reduce overfitting

Decision trees

Using **Scikit-Learn**:



Test accuracy: 66.67%

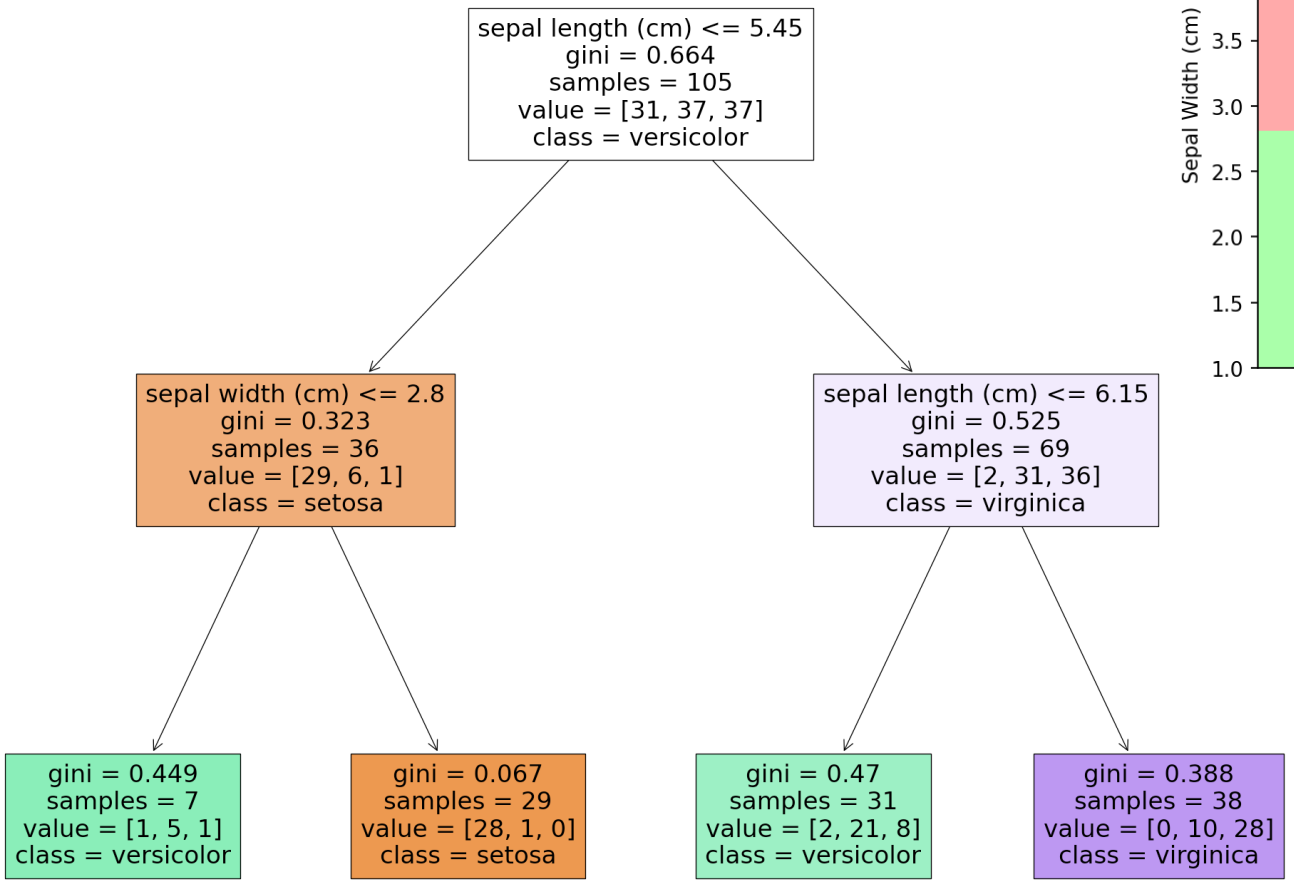
Test accuracy: 71.11%

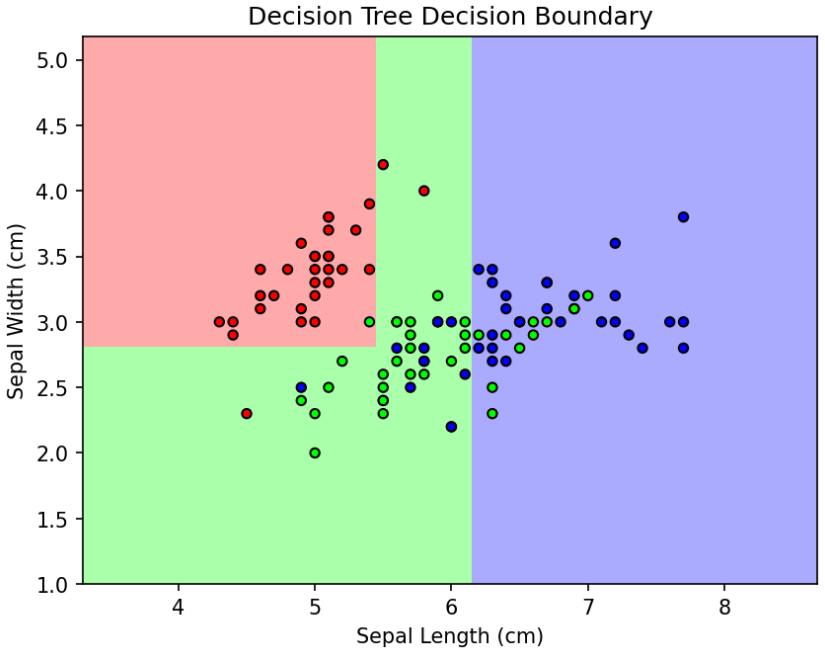
Test accuracy: 75.56%

Decision trees

```
DecisionTreeClassifier(min_samples_leaf=5, max_depth=2, random_state=42)
```

Tree visualization:





Decision trees

Hyperparameters for regularization:

`max_features`

Maximum number of features that are evaluated for splitting at each node

`max_leaf_nodes`

Maximum number of leaf nodes

`min_samples_split`

Minimum number of samples a node must have before it can be split

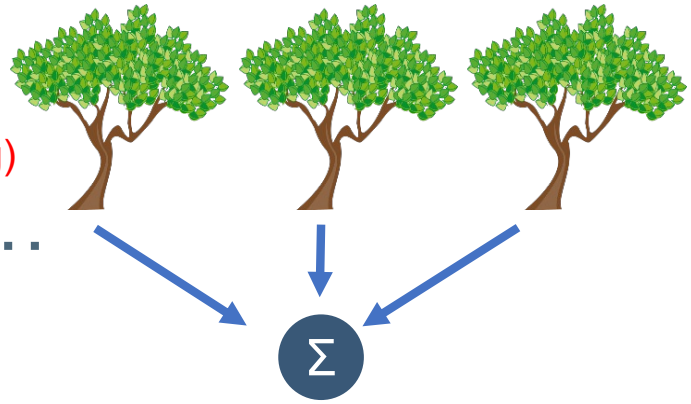
`min_samples_leaf`

Minimum number of samples a leaf node must have to be created

`min_weight_fraction_leaf`

Same as `min_samples_leaf` but expressed as a fraction of the total number of weighted instances

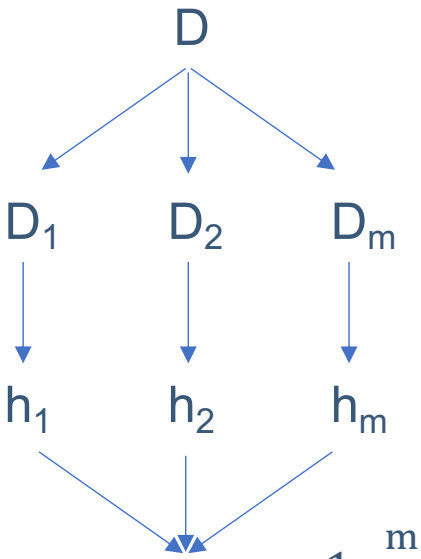
Random forest



(Single classifier > Risk of overfitting)

Lets ask many trees instead of one…

Bagging: A very effective way to reduce overfitting



(Draw m independent samples with replacement)

(Train m classifiers)

$$h(X) = \frac{1}{m}\sum_{i=1}^{m} h_i(X)$$ (Decision by majority)

D = [1, 2, 3]

D_1 = [1, 3, 1]

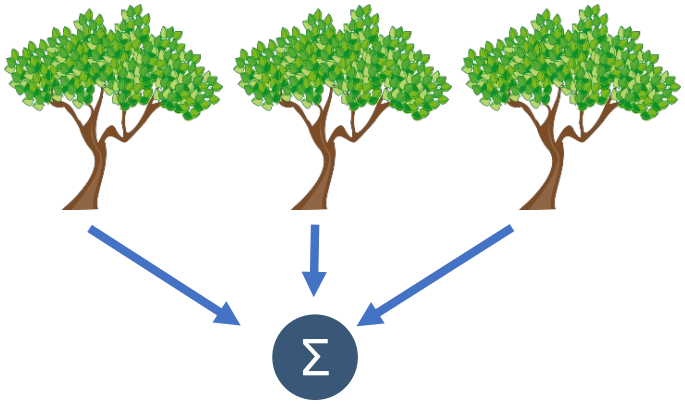
D_2 = [1, 2, 2]

⋮

Reducing variance (+ provide calibrated probability)

Random forest

Bagged decision trees (with modifications)



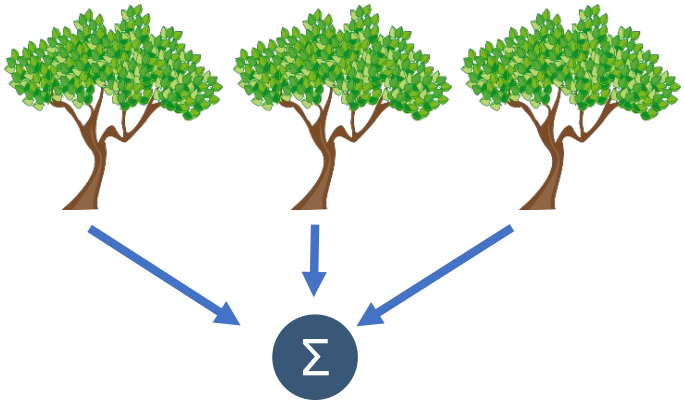
1. Sample m data sets $D_1, \ldots, D_m$ from D with replacement.
2. For each D_i train a full decision with the modification: before each split randomly subsample $k \leq d$ and only consider these features for your split (increases variance of the trees).
3. The final classifier is $h(x) = \frac{1}{m}\sum_{i=1}^{m} h_i(x)$

Random forest

Motivation

Number of features pr. split

Number of trees



Easy to run: Only two parameters **m** and **k**

(Good choice: **m** as large as you can, $\mathbf{k} = \sqrt{\mathrm{d}}$)

Requires little to no pre-processing

Deals with the variance (over-fitting) problem of decision trees

Also: Can be used as an effective method for dimensionality reduction

Random forest

Using **Scikit-Learn**:

```python
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Load the Iris dataset
iris = datasets.load_iris()
X = iris.data[:, :2]  # Using only the first two features (sepal length and sepal width)
y = iris.target

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Create a Random Forest classifier
random_forest_classifier = RandomForestClassifier(n_estimators=100, min_samples_leaf=3)

# Fit the Random Forest classifier to the training data
random_forest_classifier.fit(X_train, y_train)

# Make predictions on the test set
y_pred = random_forest_classifier.predict(X_test)

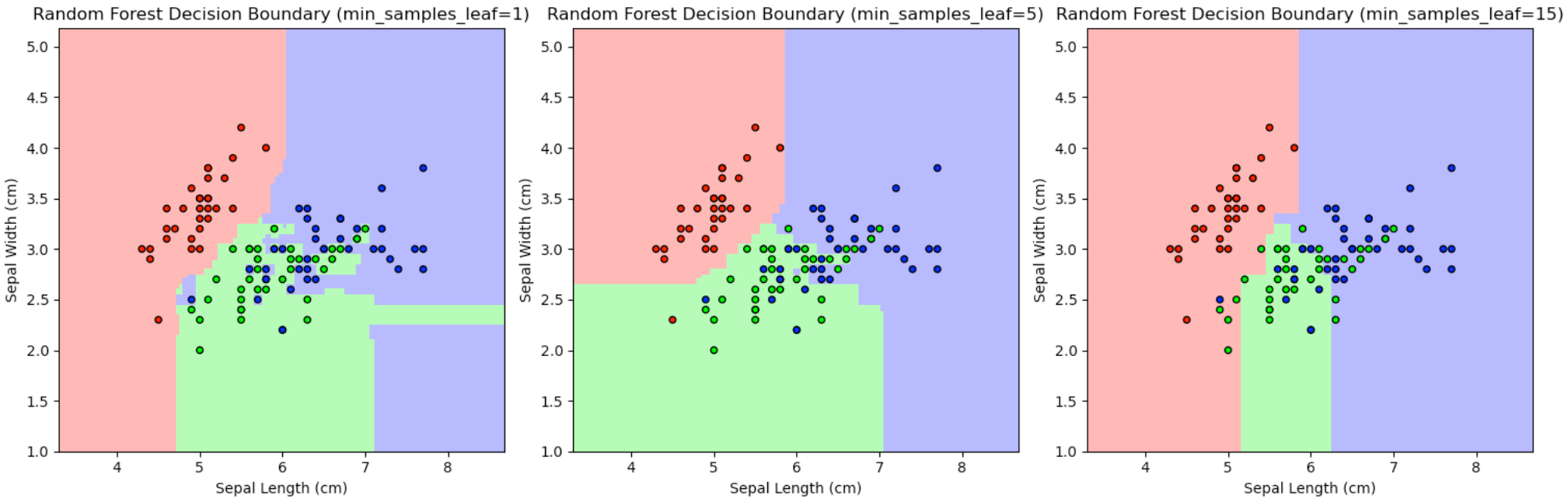
# Calculate the accuracy of the model
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy of Random Forest: {accuracy * 100:.2f}%")
```

Default: $k = \sqrt{d}$

```
Accuracy of Random Forest: 77.78%
```

Random forest

Using **Scikit-Learn**:



Test accuracy: 75.56%

Test accuracy: 77.78%

Test accuracy: 84.44%

Significantly more robust
(compared to decision trees)

Today's exercise: Who will survive?

